CI/CD Pipeline Overview

Pipeline taxonomy, release controls and source gaps.

Summary

This document explains the subject in straightforward professional English, then adds the engineering detail needed to build, operate, troubleshoot or review it. Claims are based on the repository evidence snapshot; missing source details are called out instead of guessed.

Document details

Edition 2.0 | Evidence date: 14 August 2026 | Repository: rmorampudi09-arch/Craves-Build-platform | Product evidence snapshot: main @ 3225f7fa531a6b07185fb5e034f7930f8bd8b571

Summary and how to use this document

Pipeline taxonomy, release controls and source gaps. The purpose is to make the system understandable to a new team member while still giving engineers enough detail to trace ownership, data flow, deployment impact and failure handling.

Current implementation

Direct code, README, migration, pipeline or commit evidence exists on the reviewed main snapshot.

Foundation / partial

Useful groundwork exists, but the repository does not prove a complete production runtime for every part.

Legacy / reference

Older implementation is retained for history or compatibility and is not treated as the preferred runtime.

Needs source confirmation

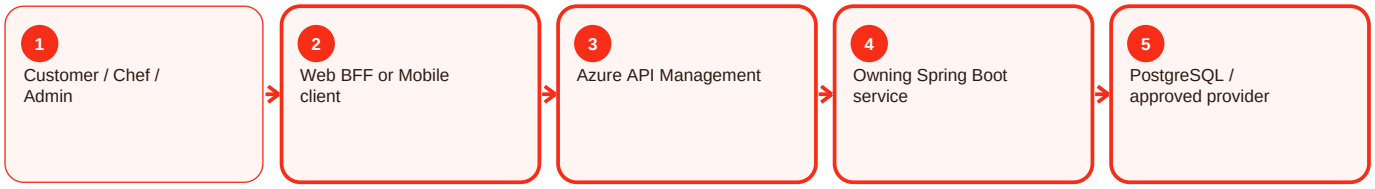
A referenced artifact or exact value could not be verified and is therefore not invented.

Reading order

- Start with the summary to understand the responsibility in normal language.
- Use process diagrams to follow requests across client, APIM, services, data and providers.
- Use the troubleshooting sections to diagnose by evidence rather than by retrying blindly.
- Use deployment and validation sections before or after changing a component.
- Use evidence status to separate proven behavior from gaps and future work.

How the request moves through Craves

The exact components depend on the feature, but the normal pattern is consistent. The user interface starts the request, the gateway and owning service enforce trust, the service writes authoritative state, and provider integrations remain behind controlled boundaries.



Key rule

A screen can hide or show an action for usability, but that is not security. APIM and the owning backend still validate the caller, role, ownership and current business state. Likewise, provider success is not accepted as Craves state until the backend verifies and records the result.

Pipeline evidence method - Overview and responsibility

Current implementation

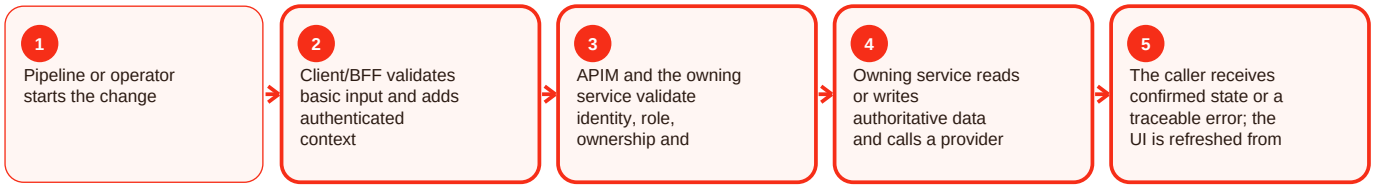
Summary

Pipeline evidence method is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For pipeline evidence method, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Pipeline evidence method - Functional behavior and data

Current implementation

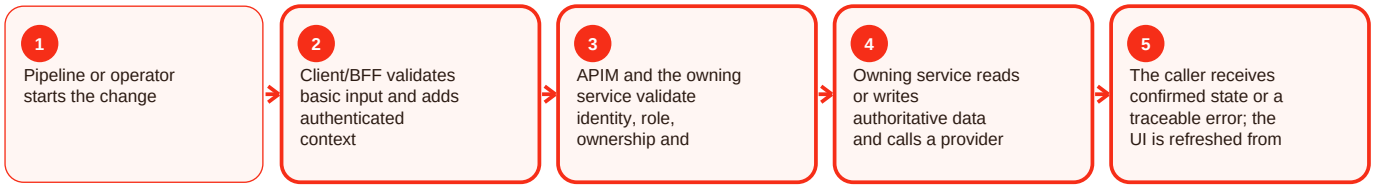
Summary

Pipeline evidence method is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For pipeline evidence method, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Pipeline evidence method - Operations, errors and deployment

Current implementation

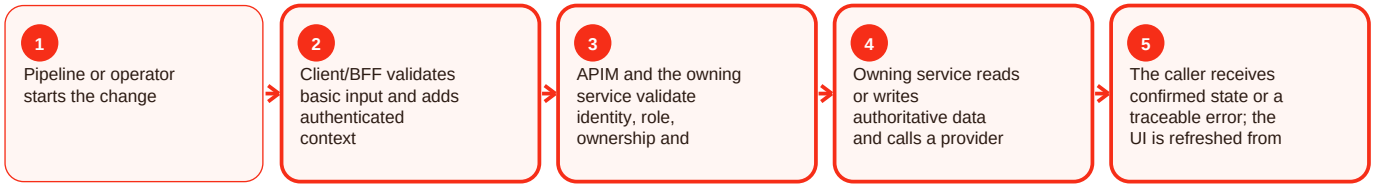
Summary

Pipeline evidence method is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For pipeline evidence method, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Backend ci - Overview and responsibility

Current implementation

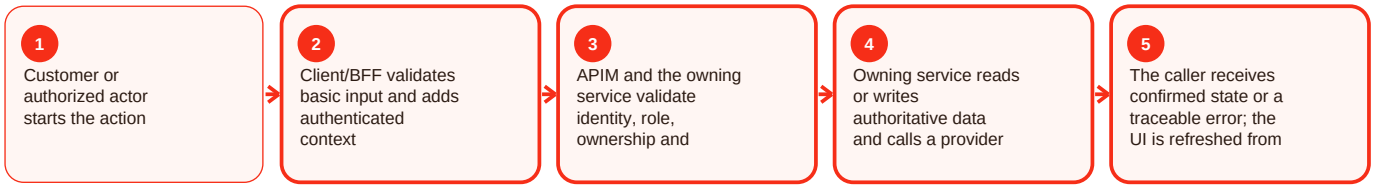
Summary

Backend ci is part of the backend area of Craves. The active backend target is seven Spring Boot services: auth, user-chef, catalog, order, subscription, integration and notification. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Services own focused business domains, validate JWT role/ownership, expose health endpoints, evolve schema through Flyway where documented, and isolate provider-specific behavior behind integration/notification boundaries. For backend CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Service health is down: Database/config/dependency/startup failure. Resolution: Inspect health/readiness and recent deployment/config changes.

Request rejected: Validation, role, ownership or state rule failed. Resolution: Use stable error code/request ID and correct the actual failing rule.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/auth-service/README.md; services/user-chef-service/README.md; services/catalog-service/README.md; services/order-service/README.md; services/subscription-service/README.md; services/integration-service/README.md; services/notification-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Backend ci - Functional behavior and data

Current implementation

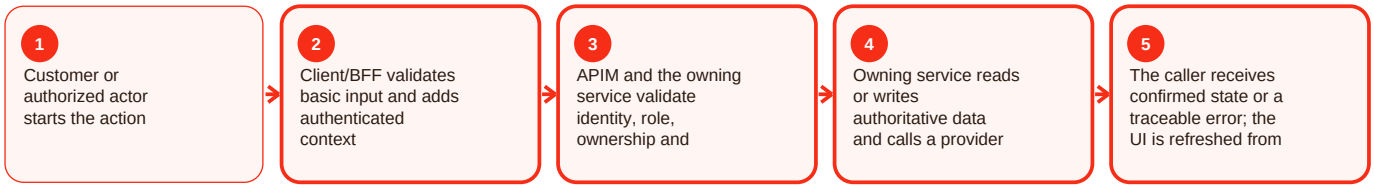
Summary

Backend ci is part of the backend area of Craves. The active backend target is seven Spring Boot services: auth, user-chef, catalog, order, subscription, integration and notification. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Services own focused business domains, validate JWT role/ownership, expose health endpoints, evolve schema through Flyway where documented, and isolate provider-specific behavior behind integration/notification boundaries. For backend CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Service health is down: Database/config/dependency/startup failure. Resolution: Inspect health/readiness and recent deployment/config changes.

Request rejected: Validation, role, ownership or state rule failed. Resolution: Use stable error code/request ID and correct the actual failing rule.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/auth-service/README.md; services/user-chef-service/README.md; services/catalog-service/README.md; services/order-service/README.md; services/subscription-service/README.md; services/integration-service/README.md; services/notification-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Backend ci - Operations, errors and deployment

Current implementation

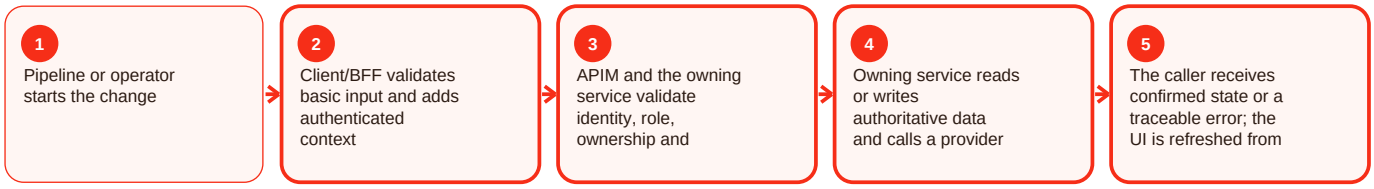
Summary

Backend ci is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For backend CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Frontend ci - Overview and responsibility

Current implementation

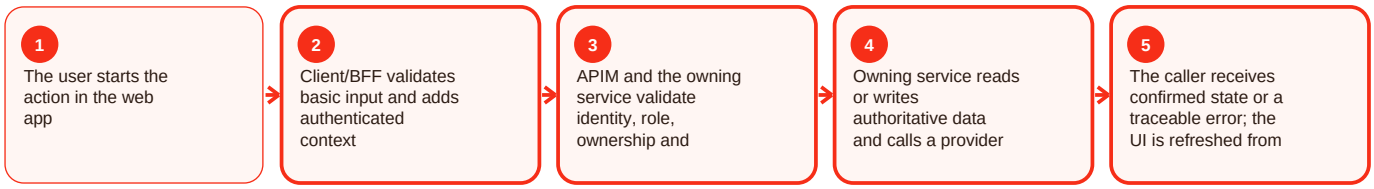
Summary

Frontend ci is part of the frontend area of Craves. The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state. For frontend CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Generic upstream error: BFF rejected malformed/unexpected backend response. Resolution: Use request ID and compare the current response contract.

Amount differs: Client state is stale. Resolution: Render backend-authoritative cart/quote state.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Frontend ci - Functional behavior and data

Current implementation

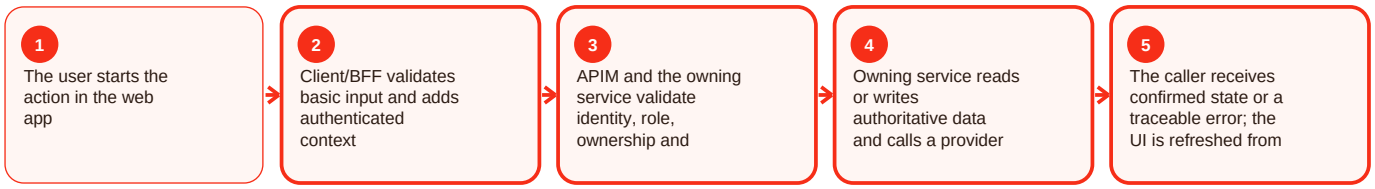
Summary

Frontend ci is part of the frontend area of Craves. The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state. For frontend CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Generic upstream error: BFF rejected malformed/unexpected backend response. Resolution: Use request ID and compare the current response contract.

Amount differs: Client state is stale. Resolution: Render backend-authoritative cart/quote state.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Frontend ci - Operations, errors and deployment

Current implementation

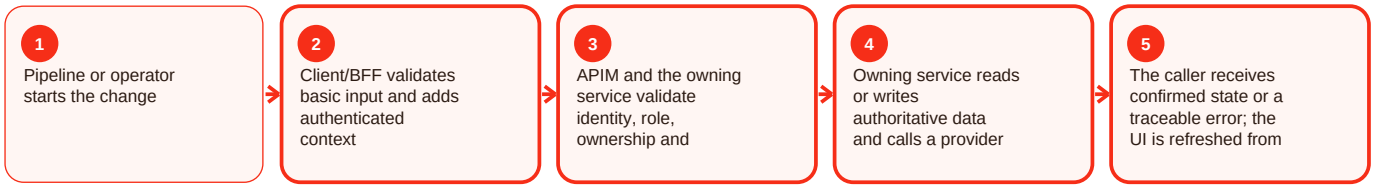
Summary

Frontend ci is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For frontend CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Admin ci - Overview and responsibility

Current implementation

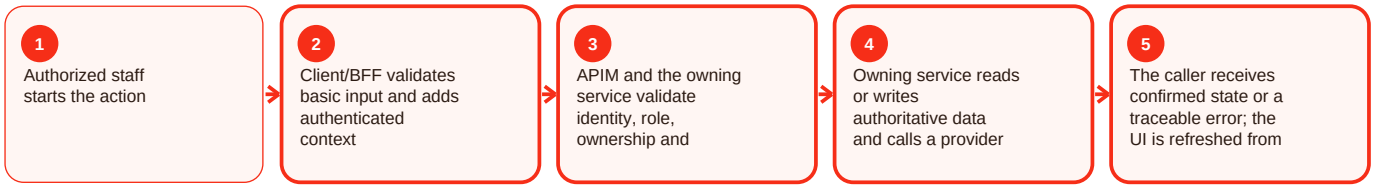
Summary

Admin ci is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Admin ci - Functional behavior and data

Current implementation

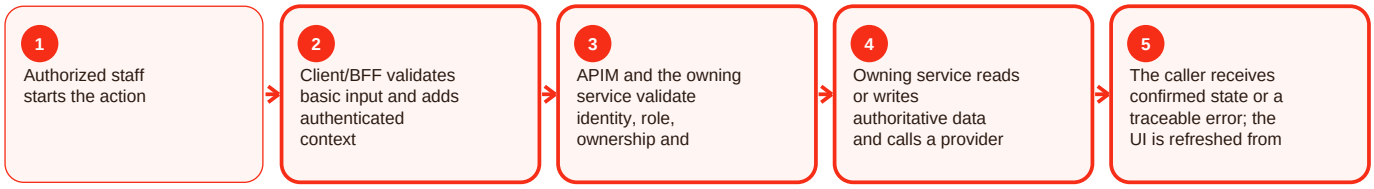
Summary

Admin ci is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Admin ci - Operations, errors and deployment

Current implementation

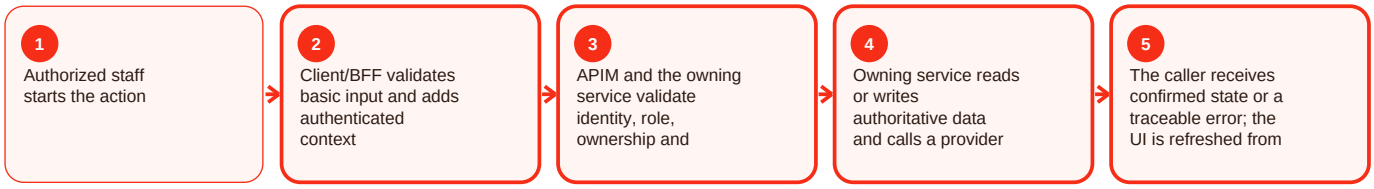
Summary

Admin ci is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Mobile ci - Overview and responsibility

Foundation / partial

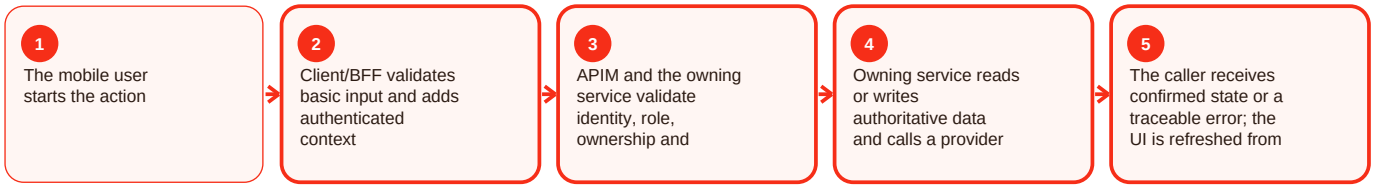
Summary

Mobile ci is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For mobile CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Mobile ci - Functional behavior and data

Foundation / partial

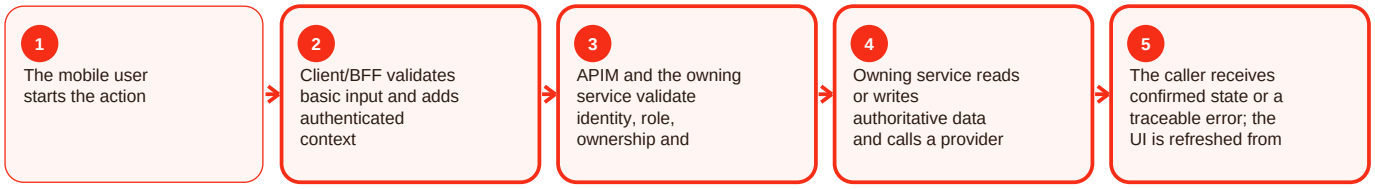
Summary

Mobile ci is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For mobile CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Mobile ci - Operations, errors and deployment

Foundation / partial

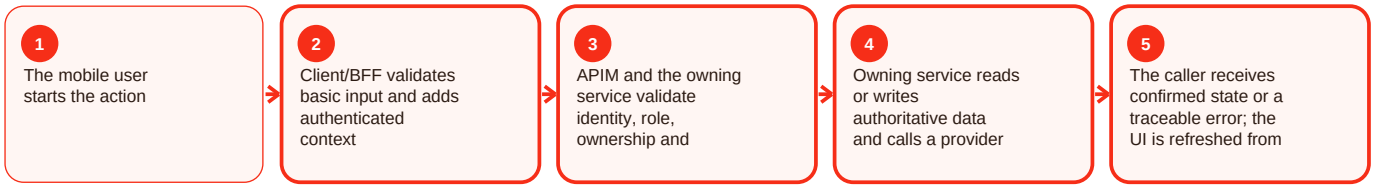
Summary

Mobile ci is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For mobile CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Apim ci - Overview and responsibility

Current implementation

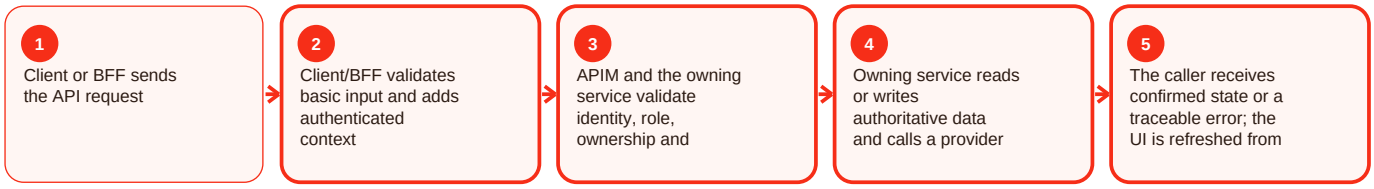
Summary

Apim ci is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For APIM CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Apim ci - Functional behavior and data

Current implementation

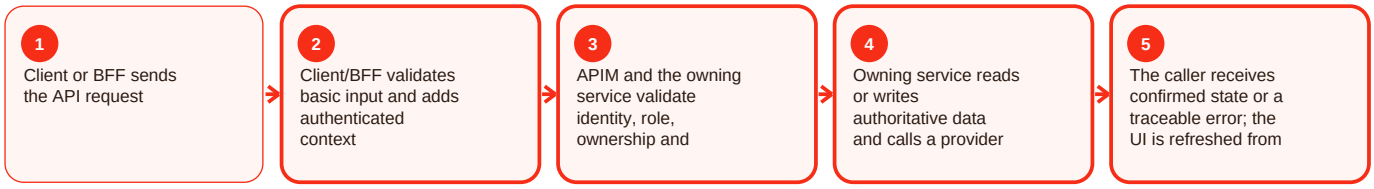
Summary

Apim ci is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For APIM CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Apim ci - Operations, errors and deployment

Current implementation

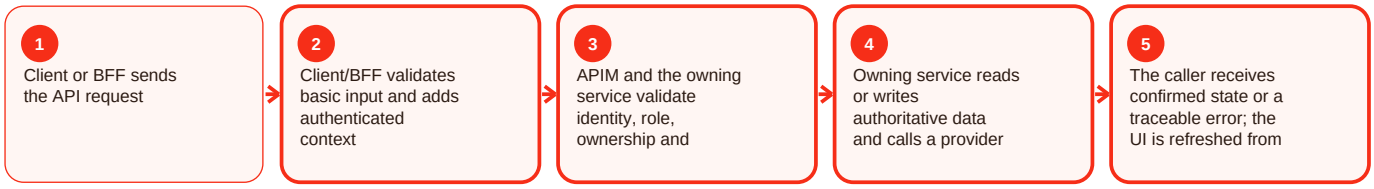
Summary

Apim ci is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For APIM CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Data migration ci - Overview and responsibility

Current implementation

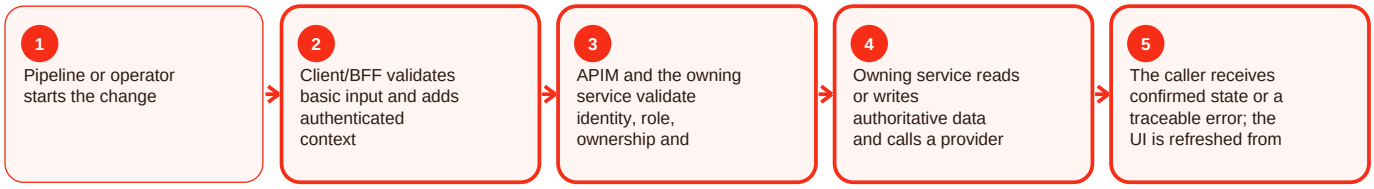
Summary

Data migration ci is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For data migration CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Data migration ci - Functional behavior and data

Current implementation

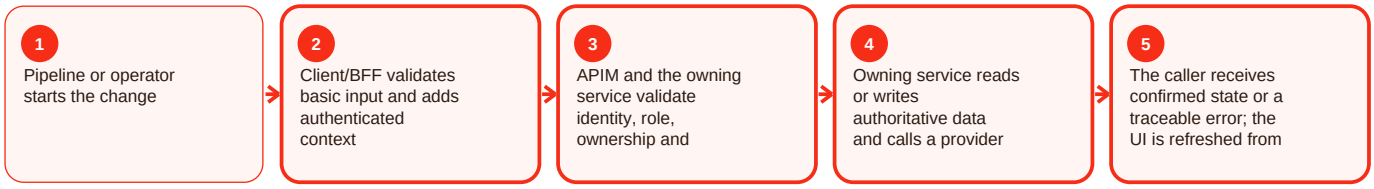
Summary

Data migration ci is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For data migration CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Data migration ci - Operations, errors and deployment

Current implementation

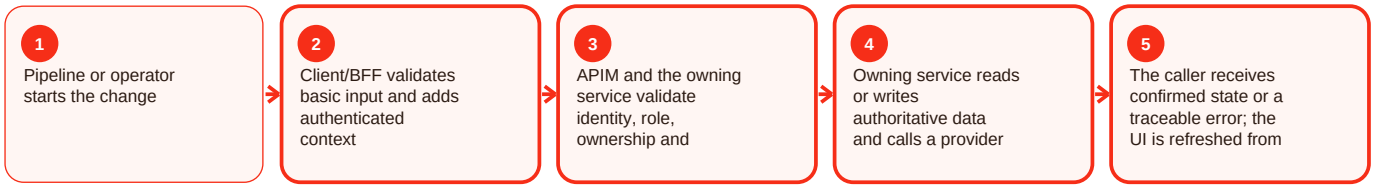
Summary

Data migration ci is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For data migration CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Privacy / security ci - Overview and responsibility

Current implementation

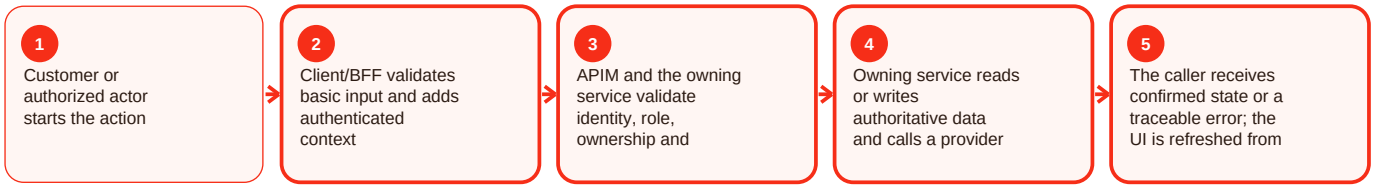
Summary

Privacy / security ci is part of the privacy area of Craves. Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent. For privacy/security CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Privacy / security ci - Functional behavior and data

Current implementation

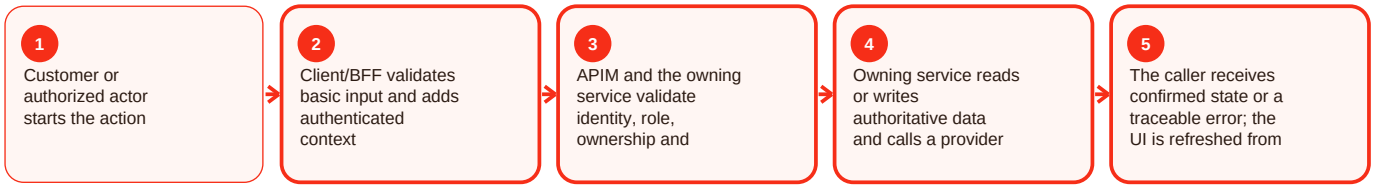
Summary

Privacy / security ci is part of the privacy area of Craves. Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent. For privacy/security CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Privacy / security ci - Operations, errors and deployment

Current implementation

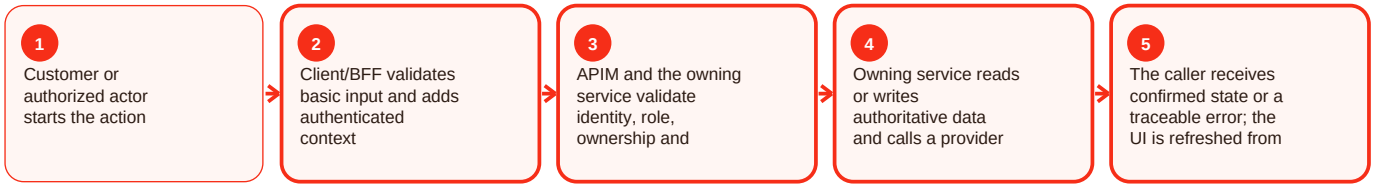
Summary

Privacy / security ci is part of the privacy area of Craves. Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent. For privacy/security CI, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Smoke / e2e - Overview and responsibility

Current implementation

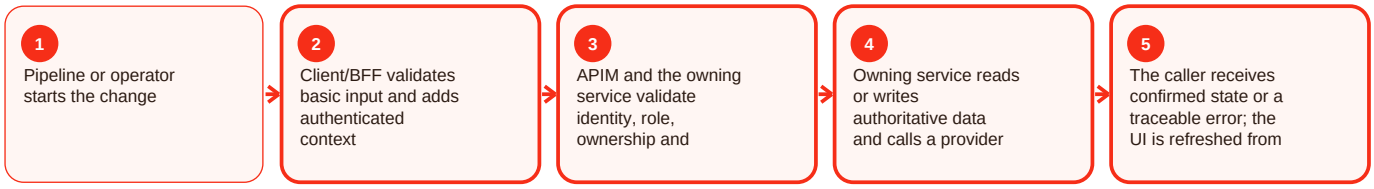
Summary

Smoke / e2e is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For smoke/e2e, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Smoke / e2e - Functional behavior and data

Current implementation

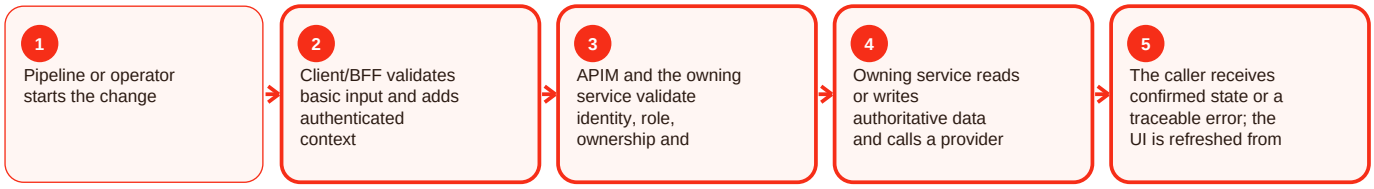
Summary

Smoke / e2e is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For smoke/e2e, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Smoke / e2e - Operations, errors and deployment

Current implementation

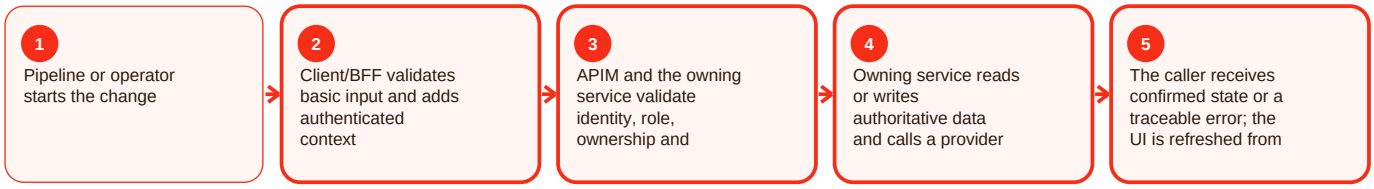
Summary

Smoke / e2e is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For smoke/e2e, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Release gates - Overview and responsibility

Current implementation

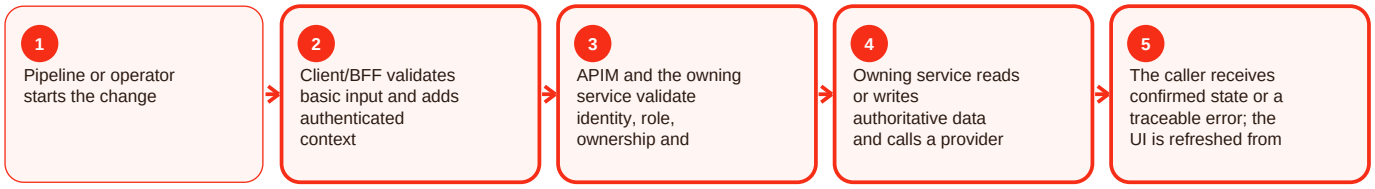
Summary

Release gates is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For release gates, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Release gates - Functional behavior and data

Current implementation

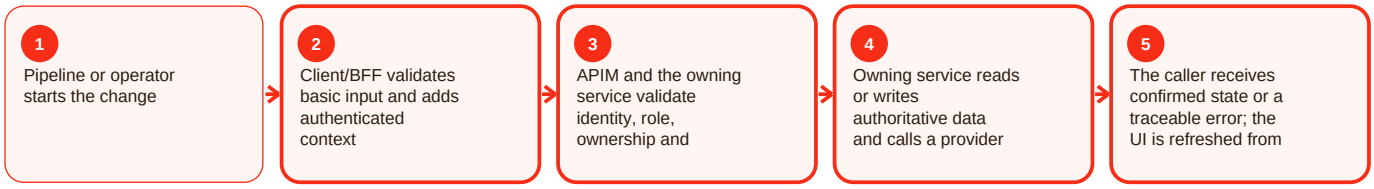
Summary

Release gates is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For release gates, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Release gates - Operations, errors and deployment

Current implementation

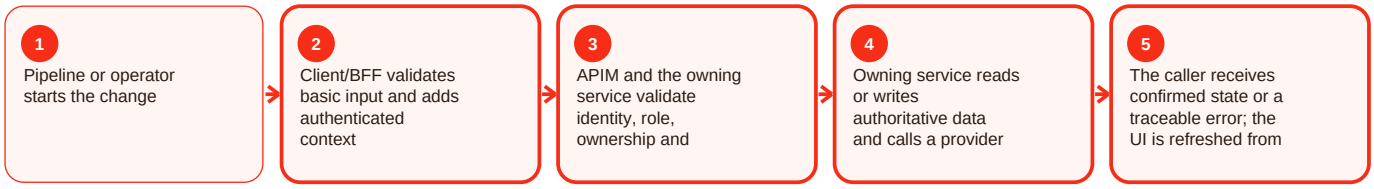
Summary

Release gates is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For release gates, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Pipeline path gaps - Overview and responsibility

Needs source confirmation

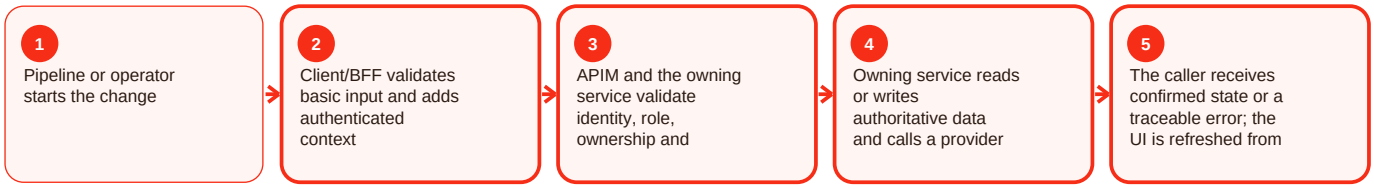
Summary

Pipeline path gaps is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For pipeline path gaps, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

The source reference could not be verified at the cited path, so this document does not invent the missing detail. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Pipeline path gaps - Functional behavior and data

Needs source confirmation

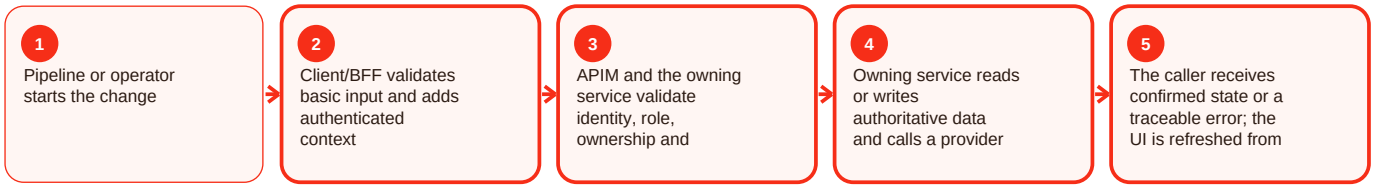
Summary

Pipeline path gaps is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For pipeline path gaps, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

The source reference could not be verified at the cited path, so this document does not invent the missing detail. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Pipeline path gaps - Operations, errors and deployment

Needs source confirmation

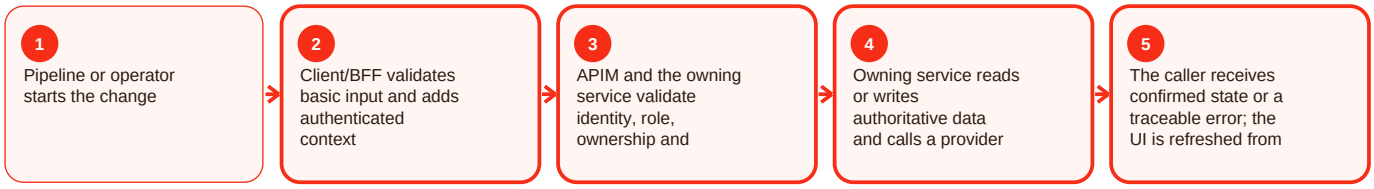
Summary

Pipeline path gaps is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For pipeline path gaps, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

The source reference could not be verified at the cited path, so this document does not invent the missing detail. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Container build - Overview and responsibility

Current implementation

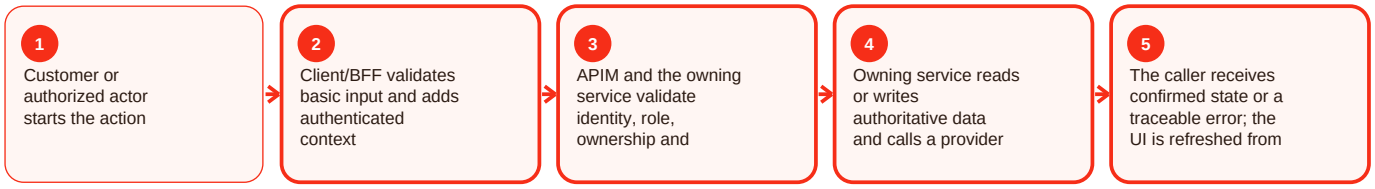
Summary

Container build is part of the ai area of Craves. Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions. For container build, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Container build - Functional behavior and data

Current implementation

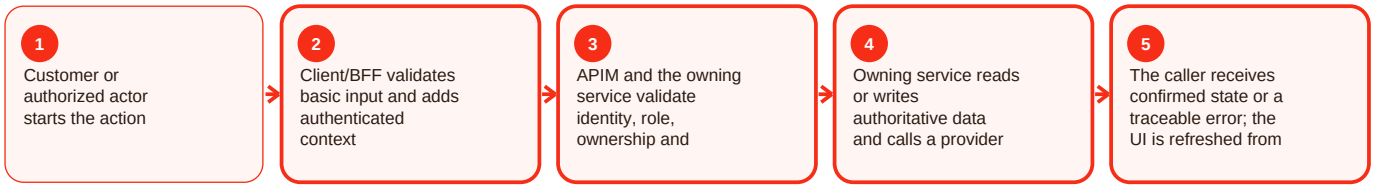
Summary

Container build is part of the ai area of Craves. Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions. For container build, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Container build - Operations, errors and deployment

Current implementation

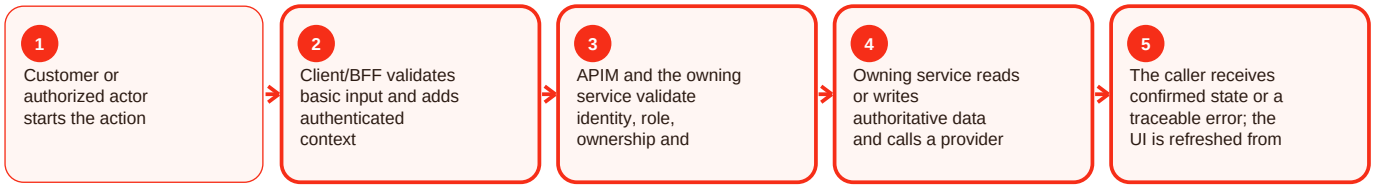
Summary

Container build is part of the ai area of Craves. Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions. For container build, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Acr push - Overview and responsibility

Current implementation

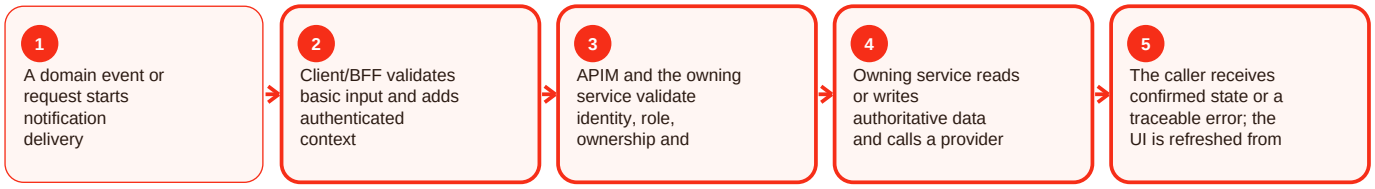
Summary

Acr push is part of the notification area of Craves. Notifications centralize email, SMS and push so domain services do not each implement vendor logic. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

notification-service documents Azure Communication Services, FCM, preferences, device tokens, optional Service Bus topic craves-domain-events, event-id idempotency, correlation propagation and a seven-day resend worker. For ACR push, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

External channel stays pending: Provider adapter is disabled or unavailable. Resolution: Check channel configuration and delivery-attempt state.

Duplicate delivery: Retry did not reuse the event/request key. Resolution: Use idempotency evidence before resending.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/notification-service/README.md; apps/customer-web-next/src/components/notifications/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Acr push - Functional behavior and data

Current implementation

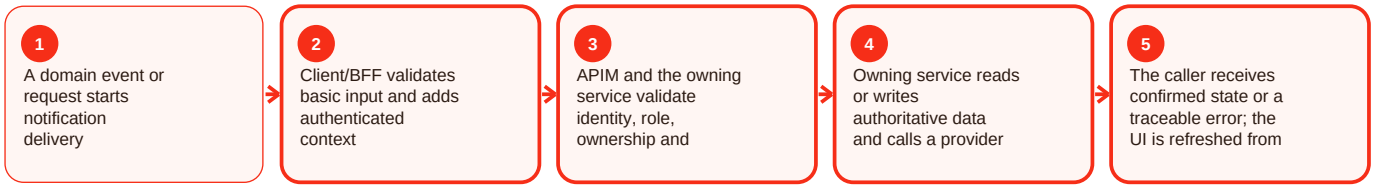
Summary

Acr push is part of the notification area of Craves. Notifications centralize email, SMS and push so domain services do not each implement vendor logic. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

notification-service documents Azure Communication Services, FCM, preferences, device tokens, optional Service Bus topic craves-domain-events, event-id idempotency, correlation propagation and a seven-day resend worker. For ACR push, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

External channel stays pending: Provider adapter is disabled or unavailable. Resolution: Check channel configuration and delivery-attempt state.

Duplicate delivery: Retry did not reuse the event/request key. Resolution: Use idempotency evidence before resending.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/notification-service/README.md; apps/customer-web-next/src/components/notifications/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Acr push - Operations, errors and deployment

Current implementation

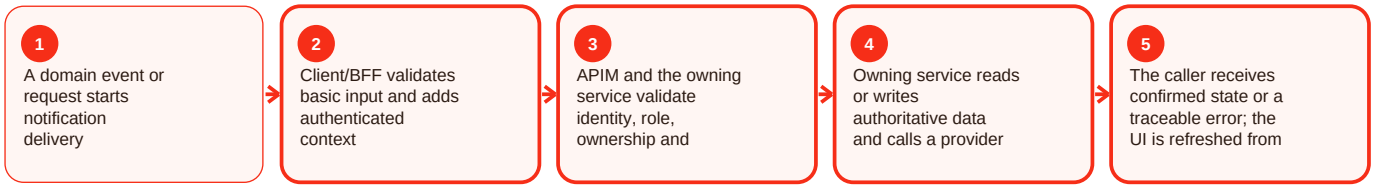
Summary

Acr push is part of the notification area of Craves. Notifications centralize email, SMS and push so domain services do not each implement vendor logic. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

notification-service documents Azure Communication Services, FCM, preferences, device tokens, optional Service Bus topic craves-domain-events, event-id idempotency, correlation propagation and a seven-day resend worker. For ACR push, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

External channel stays pending: Provider adapter is disabled or unavailable. Resolution: Check channel configuration and delivery-attempt state.

Duplicate delivery: Retry did not reuse the event/request key. Resolution: Use idempotency evidence before resending.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/notification-service/README.md; apps/customer-web-next/src/components/notifications/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Container app revision - Overview and responsibility

Current implementation

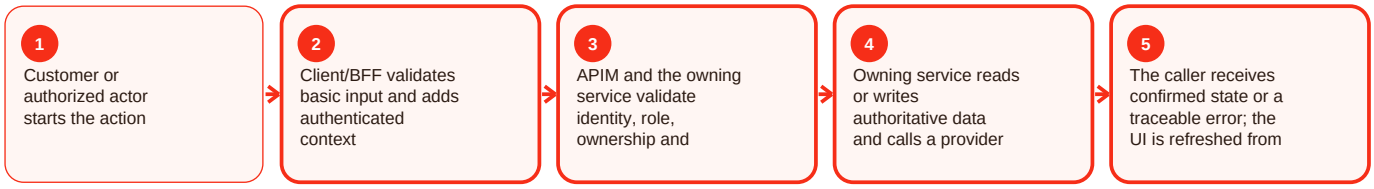
Summary

Container app revision is part of the ai area of Craves. Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions. For Container App revision, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Container app revision - Functional behavior and data

Current implementation

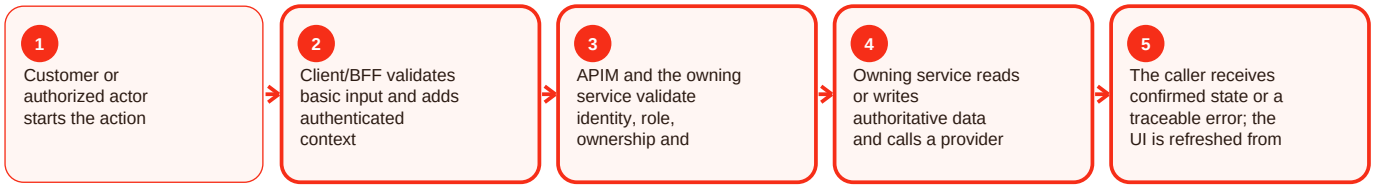
Summary

Container app revision is part of the ai area of Craves. Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions. For Container App revision, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Container app revision - Operations, errors and deployment

Current implementation

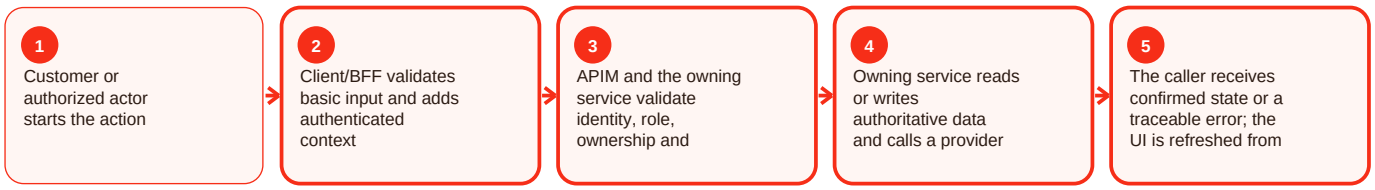
Summary

Container app revision is part of the ai area of Craves. Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions. For Container App revision, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Readiness - Overview and responsibility

Current implementation

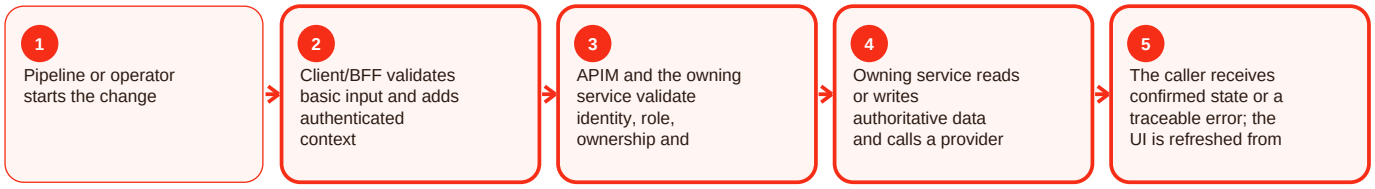
Summary

Readiness is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For readiness, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Readiness - Functional behavior and data

Current implementation

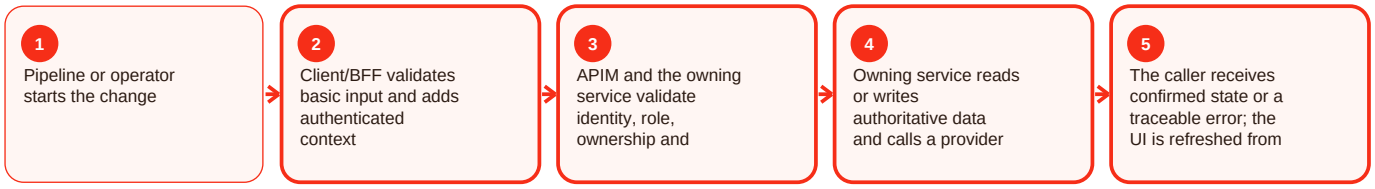
Summary

Readiness is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For readiness, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Readiness - Operations, errors and deployment

Current implementation

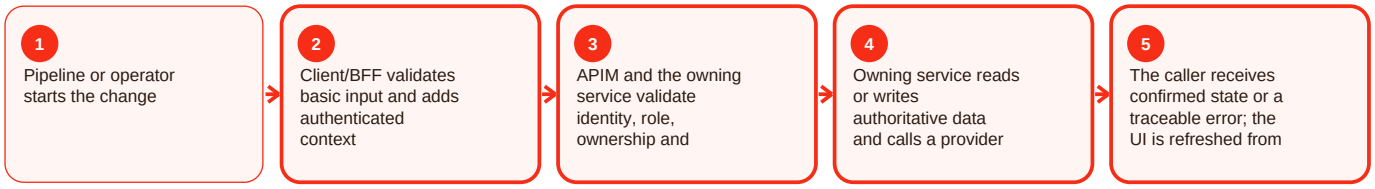
Summary

Readiness is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For readiness, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Rollback - Overview and responsibility

Current implementation

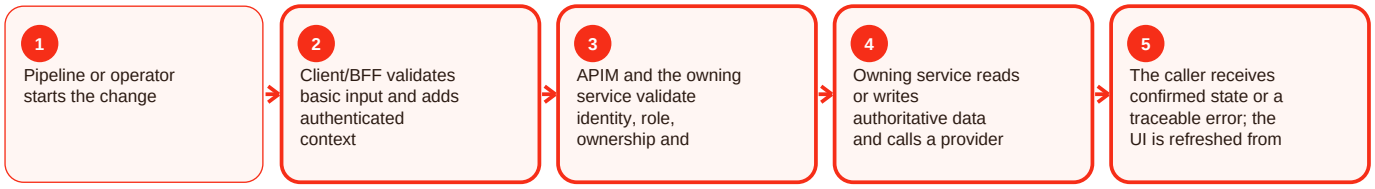
Summary

Rollback is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For rollback, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Rollback - Functional behavior and data

Current implementation

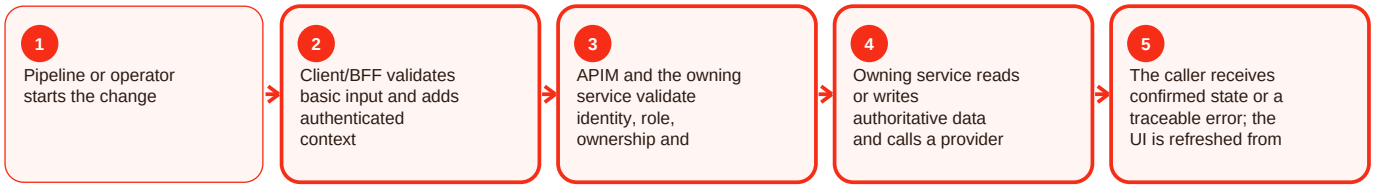
Summary

Rollback is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For rollback, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Rollback - Operations, errors and deployment

Current implementation

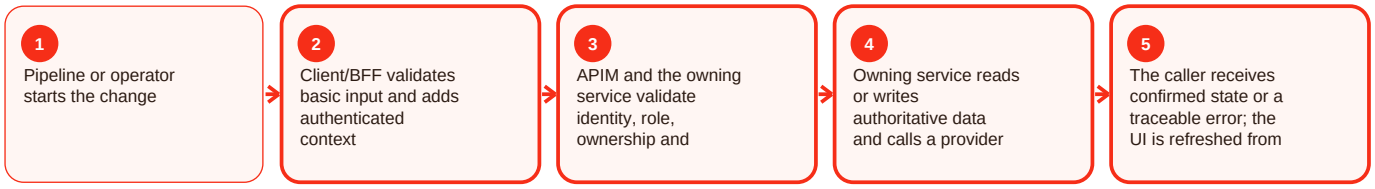
Summary

Rollback is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For rollback, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Deployment markers - Overview and responsibility

Current implementation

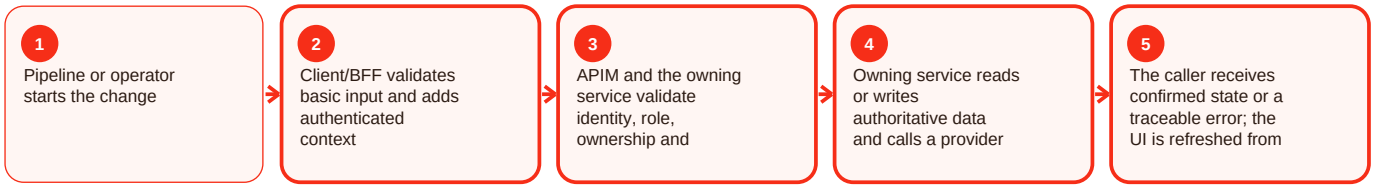
Summary

Deployment markers is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For deployment markers, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Deployment markers - Functional behavior and data

Current implementation

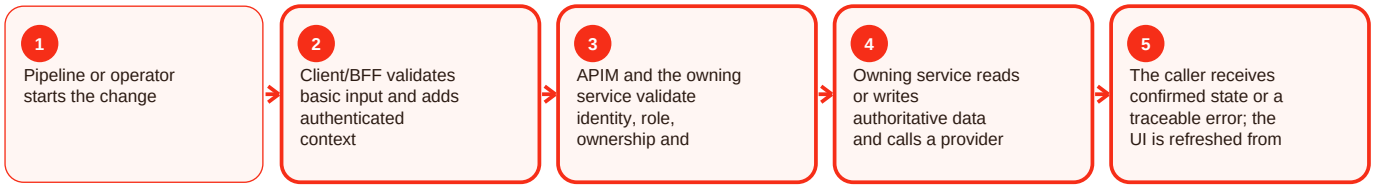
Summary

Deployment markers is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For deployment markers, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Deployment markers - Operations, errors and deployment

Current implementation

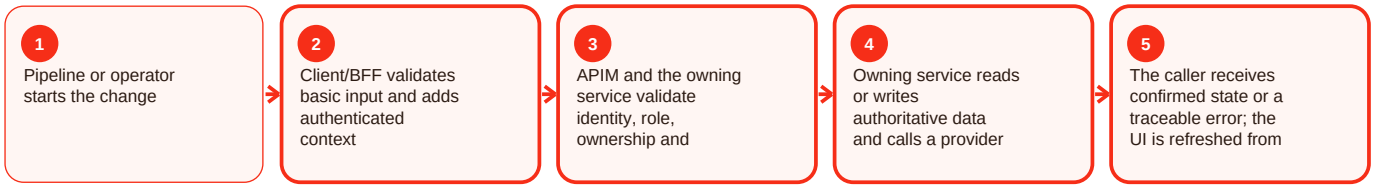
Summary

Deployment markers is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For deployment markers, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Environment variables - Overview and responsibility

Current implementation

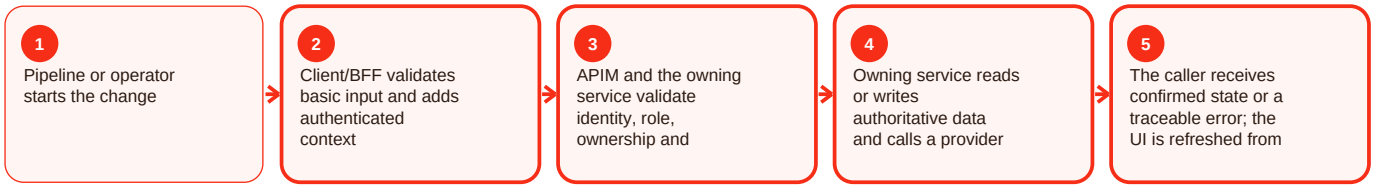
Summary

Environment variables is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For environment variables, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Operational checklist and documented gaps

Before making a change

- Confirm the intended actor and environment.
- Confirm the current authoritative state before changing anything.
- Capture HTTP status, stable error code and request/correlation ID without secrets.
- Validate the owning component health and required dependency/configuration.
- After the action, reload the state from the backend and confirm the expected result.
- Confirm the deployed artifact/revision and readiness.
- If rollback is used, verify the previous revision is healthy before closing the change.

Repository gaps that must stay visible

- APIM README cites infra/apim/craves-openapi.yaml, but that literal artifact was not resolvable on main during this evidence snapshot.
- customer-web-next README names azure-pipelines-customer-web-next.yml, but that literal root path was not resolvable; deployment behavior described by the README is retained while trigger/variable details are not invented.
- Native mobile has strong milestone identity/API/CI evidence, but a clearly complete runnable React Native application was not established on the reviewed main snapshot.
- Legacy preview URLs prove historical deployment references rather than current production routing.
- Commercial values such as commissions, payout percentages, subscription pricing, catering thresholds and SLAs are not fabricated where source is silent.

Definition of done

A change is complete only when the intended behavior works, authorization still fails closed, authoritative data is correct, health/readiness are good, monitoring or correlation evidence is available, the rollback path remains valid, and the documentation has been updated if the contract or operating procedure changed.